



DogMate

The Smart Way to Care, Connect & Walk Together

Maintenance Guide

Developer Reference for Setup, Configuration & Code Maintenance

Table of Contents

1. Project Overview	—
1.1 Purpose of the Application	—
1.2 Current Architecture at a Glance	—
1.3 Technology Stack	—
2. Environment Setup	—
2.1 Prerequisites	—
2.2 Cloning and First Build	—
2.3 Environment Variables	—
3. Database Configuration	—
3.1 Default: PostgreSQL via Supabase	—
3.2 Connection Pooling	—
3.3 JPA / Hibernate DDL Modes	—
4. Switching Between External DB and In-Memory	—
4.1 Why the Switch Is Possible	—
4.2 Implementing an In-Memory Repository	—
4.3 Activating an In-Memory Profile	—
5. Feature-by-Feature Code Map	—
5.1 Authentication & Registration	—
5.2 Dog Profile Management	—
5.3 Vaccinations	—
5.4 Medications	—
5.5 Food Inventory	—
5.6 Reminders	—
5.7 Walks, Proximity & Ping	—
5.8 Dog Walker Profile & Ratings	—
5.9 Emergency SOS — Clinic Finder	—
6. Cache Management	—
7. Security & CORS	—
8. Testing	—
9. Common Maintenance Tasks	—

Chapter 1

Project Overview

1.1 Purpose of the Application

DogMate is a mobile application that brings together every aspect of dog ownership and professional dog walking into a single platform. The product serves two distinct audiences: private dog owners, who use the app to manage their dog's health records, food inventory, and social walks, and professional dog walkers, who manage their availability, service areas, and client reviews from the same app.

This document is the technical companion to the User Guide. It does not describe what the application does from a user perspective — it describes how the backend is structured, where each feature lives in code, how to configure the project for different environments, and how to perform common maintenance tasks. The intended reader is a developer joining or continuing work on the DogMate backend.

1.2 Current Architecture at a Glance

The backend follows a clean four-layer architecture that cleanly separates HTTP concerns from business logic and persistence. Each layer has a single responsibility and only depends on the layer directly below it.

Layer	Responsibility	Example Class
Controller	Handles HTTP requests and responses. Maps URL paths to service calls.	UserController.java
Service	Orchestrates business logic. Coordinates between repositories and domain entities.	UserService.java
Domain	Holds JPA entities and core business rules. Pure Java, no Spring annotations beyond JPA.	RegularUser.java
Infrastructure	JPA repository implementations. Talks directly to the database.	UserRepository.java

Repositories are exposed to the service layer through interfaces (for example `IUserRepository`, `IDogRepository`). This is the key design decision behind the project — it makes the persistence implementation pluggable. The current implementation uses Spring Data JPA against PostgreSQL, but the same interfaces can be implemented with an in-memory collection for local development or

testing. The pattern is documented in detail in Chapter 4.

Source Tree Preview

```
Server/DogMate/
+-- pom.xml <- Maven build, Spring Boot 3.5.7, Java 17
+-- .env.example <- Sample environment variables
+-- src/main/
| +-- java/com/DogMate/
| | +-- DogMateApplication.java <- Application entry point
| | +-- Config/ <- Beans: CacheConfig, SecurityConfig, WebSocketConfig
| | +-- Controller/ <- 15 REST controllers under /api/*
| | +-- DTO/ <- Request and response payloads
| | +-- Domain/ <- JPA entities (UserAccount, Dog, Vaccination, ...)
| | +-- Service/ <- Business logic + I*Repository interfaces
| | +-- Infrastructure/ <- JPA repository implementations
| | +-- util/ <- Helpers (PhoneValidation, etc.)
| +-- resources/
| +-- application.properties <- All runtime configuration
| +-- db/manual/ <- SQL scripts for manual DB migrations
+-- src/test/ <- JUnit + Selenium tests
```

1.3 Technology Stack

Component	Technology
Language	Java 17
Framework	Spring Boot 3.5.7
Build tool	Maven (wrapper included: mvnw / mvnw.cmd)
Database	PostgreSQL 15+ (hosted on Supabase by default)
Persistence	Spring Data JPA + Hibernate
Cache	Caffeine (in-memory, 5-second TTL)
Security	Spring Security with BCrypt password hashing
Real-time	WebSocket (STOMP) — used for Ping notifications
Mail	Spring Boot Mail (Gmail SMTP) — verification codes
Testing	JUnit 5, Selenium 4.43, Spring Boot Test

Note: The frontend (iOS mobile app) is not covered in this guide. This document focuses exclusively on the Spring Boot backend located under the `Server/` directory of the repository.

Chapter 2

Environment Setup

This chapter walks through the steps required to get the backend running on a fresh development machine — from cloning the repository to launching the server for the first time.

2.1 Prerequisites

- Java Development Kit (JDK) 17 or higher.
- Maven 3.6 or higher (the Maven wrapper mvnw is included and recommended).
- Git, for cloning the repository.
- A PostgreSQL database (Supabase recommended for parity with production).
- Any IDE with Spring Boot support — IntelliJ IDEA is the configured IDE in the repository (.idea/ files are checked in).

2.2 Cloning and First Build

1	Clone the repository git clone https://github.com/oriadika/DogMate-The-Smart-Way-to-Care-Connect-and-Walk-Together.git
2	Navigate to the server directory cd DogMate-The-Smart-Way-to-Care-Connect-and-Walk-Together/Server/DogMate
3	Copy the environment template cp .env.example .env and fill in your database credentials.
4	Build the project ./mvnw clean install — downloads dependencies and compiles the project.
5	Run the application ./mvnw spring-boot:run — starts the server on port 8080 by default.
6	Verify the server Open http://localhost:8080/ in a browser. The HomeController returns a basic health endpoint.

2.3 Environment Variables

All sensitive configuration is read from environment variables (or the .env file in development). Defaults are set in src/main/resources/application.properties — these defaults are not safe for production and exist only to enable a quick first run.

```
.env

# Database
SPRING_DATASOURCE_URL=jdbc:postgresql://<host>:5432/postgres?sslmode=require
SPRING_DATASOURCE_USERNAME=postgres
SPRING_DATASOURCE_PASSWORD=<your-db-password>
SPRING_DATASOURCE_POOL_SIZE=10
SPRING_DATASOURCE_MIN_IDLE=2

# Outgoing mail (verification codes, support requests)
SPRING_MAIL_HOST=smtp.gmail.com
SPRING_MAIL_PORT=587
SPRING_MAIL_USERNAME=<sender-address>
SPRING_MAIL_PASSWORD=<gmail-app-password>
DOGMATE_MAIL_FROM=<sender-address>
```

Important: The repository currently contains a hard-coded mail App Password and database password in application.properties as fallback defaults. These should be removed before any public release and replaced with environment-variable-only configuration.

Chapter 3

Database Configuration

The application is built against PostgreSQL and uses Spring Data JPA with Hibernate as the persistence provider. All database settings live in a single file: `src/main/resources/application.properties`.

3.1 Default: PostgreSQL via Supabase

The default configuration connects to a Supabase-hosted PostgreSQL instance through the transaction pooler on port 6543. The transaction pooler multiplexes connections, which avoids exhausting session slots on startup — critical when the application is restarted frequently during development.

```
# application.properties (defaults)
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:postgresql://...:6543/postgres?sslmode=require}
spring.datasource.driverClassName=org.postgresql.Driver
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:postgres.xdaxkncdjhlqlupaxuzd}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:<password>}
```

3.2 Connection Pooling

HikariCP (the default Spring Boot connection pool) is configured with a maximum of 10 connections and 2 idle. Under concurrent HTTP load, lowering this value caused "Connection is not available" errors on the pooled endpoint — keep the defaults unless the database tier is upgraded.

```
spring.datasource.hikari.maximum-pool-size=${SPRING_DATASOURCE_POOL_SIZE:10}
spring.datasource.hikari.minimum-idle=${SPRING_DATASOURCE_MIN_IDLE:2}
```

3.3 JPA / Hibernate DDL Modes

Hibernate is currently configured with `ddl-auto=update`, which means it will automatically create missing tables and columns based on the JPA entity classes. This is convenient during active development but is not safe for production — schema changes should be reviewed and applied through manual SQL scripts (see `src/main/resources/db/manual/`).

```
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update # change to "validate" for production
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.format_sql=true
```

Tip: For production deployments, switch `ddl-auto` to "validate" and apply schema migrations manually using the SQL files under `src/main/resources/db/manual/`. This forces explicit, reviewable changes to the database schema.

Chapter 4

Switching Between External DB and In-Memory

One of the most important design decisions in the DogMate backend is the separation between repository interfaces and their implementations. This chapter explains how to leverage that separation to swap the live PostgreSQL backend for an in-memory implementation — useful for local development without a database, fast integration tests, or demo environments.

4.1 Why the Switch Is Possible

Every repository in the project is split into two parts:

- An interface in the Service package (for example IUserRepository, IDogRepository, IFoodStockRepository) — defines the contract.
- A JPA implementation in the Infrastructure package (UserRepository, DogRepository, FoodStockRepository) — extends JpaRepository and the matching interface.

Services depend on the interface, never on the concrete JPA class. This means a second implementation of the same interface — backed by an in-memory data structure — can be introduced without changing a single line of service code.

Existing Repository Interfaces

```
com.DogMate.Service.IUserRepository
com.DogMate.Service.IDogRepository
com.DogMate.Service.IFoodStockRepository
com.DogMate.Service.IReminderRepository
com.DogMate.Service.IDogWalkerRepository
com.DogMate.Service.IDogWalkerRatingRepository
com.DogMate.Service.IDogRelationshipRepository
com.DogMate.Service.IDogEventRepository
com.DogMate.Service.IDogDocumentRepository
com.DogMate.Service.IVerificationCodeRepository
com.DogMate.Service.IConversationRepository
com.DogMate.Service.ISniffRequestRepository
com.DogMate.Service.ISupportRequestRepository
com.DogMate.Service.IVeterinaryClinicRepository
com.DogMate.Service.IEmergencyContactRepository
```

4.2 Implementing an In-Memory Repository

An in-memory repository is a plain Java class that implements the `IDogRepository` interface using a `ConcurrentHashMap` (or any thread-safe collection). The example below shows a complete in-memory implementation of `IDogRepository`. The same pattern applies to every other interface.

```
package com.DogMate.Infrastructure;

import com.DogMate.Domain.Dog;
import com.DogMate.Service.IDogRepository;
import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Repository;

import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

@Repository
@Profile("memory") // only active when the "memory" profile is on
public class InMemoryDogRepository implements IDogRepository {

    private final Map<UUID, Dog> store = new ConcurrentHashMap<>();

    @Override
    public Dog save(Dog dog) {
        if (dog.getID() == null) {
            // mimic JPA-generated UUID
            dog.setID(UUID.randomUUID());
        }
        store.put(dog.getID(), dog);
        return dog;
    }

    @Override
    public Optional<Dog> findById(UUID id) {
        return Optional.ofNullable(store.get(id));
    }

    @Override
    public void deleteById(UUID id) { store.remove(id); }

    @Override
    public boolean existsById(UUID id) { return store.containsKey(id); }

    @Override
```

```
public List<Dog> findAll() { return new ArrayList<>(store.values()); }
```

```
@Override
```

```
public List<Dog> searchDogs(String text) {  
    String q = text == null ? "" : text.toLowerCase();  
    return store.values().stream()  
        .filter(d -> d.getName().toLowerCase().contains(q)  
            || d.getBreed().toLowerCase().contains(q))  
        .toList();  
}
```

```
@Override
```

```
public List<Dog> findAllForRegularUser(UUID userId) {  
    return store.values().stream()  
        .filter(d -> d.getDogRelationships().stream()  
            .anyMatch(r -> r.getRegularUser().getId().equals(userId)))  
        .toList();  
}  
}
```

Important: Disabling the JPA Bean

The existing JPA repository (DogRepository) is automatically discovered by Spring Data JPA and registered as a bean. To use the in-memory version exclusively, the JPA repository must be excluded from the "memory" profile. Add a `@Profile("!memory")` annotation, or move the JPA repositories into a package that is conditionally scanned via `@EnableJpaRepositories` on a configuration class active only when the memory profile is off.

```
// Option A: Annotate each JPA repository
@Repository
@Profile("!memory")
public interface DogRepository extends JpaRepository<Dog, UUID>, IDogRepository {
// ...
}

// Option B: Conditional repository scan in a @Configuration class
@Configuration
@Profile("!memory")
@EnableJpaRepositories(basePackages = "com.DogMate.Infrastructure")
public class JpaPersistenceConfig { }
```

4.3 Activating an In-Memory Profile

Once the in-memory implementations exist and the JPA beans are excluded from the "memory" profile, the switch is controlled by a single Spring Boot setting. Activate the profile in any of the standard ways:

```
# 1. Via command line
./mvnw spring-boot:run -Dspring-boot.run.profiles=memory

# 2. Via application.properties
spring.profiles.active=memory

# 3. Via environment variable
export SPRING_PROFILES_ACTIVE=memory
```

Important: When the memory profile is active, the JPA auto-configuration will still try to connect to a database unless explicitly disabled. Add the following to `application-memory.properties` to skip JPA auto-configuration entirely: `spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfiguration`

Tip: For tests, use `@ActiveProfiles("memory")` on the test class instead of configuring profiles globally. This isolates in-memory storage to individual tests and keeps production configuration untouched.

Chapter 5

Feature-by-Feature Code Map

This chapter is the reference index for the application. For every feature visible to the end user, the table below lists exactly where it lives in code — which controller exposes the endpoints, which service contains the business logic, which repository interface and implementation handle persistence, and which domain entity it operates on. Use this section as the entry point when investigating a bug or extending a feature.

5.1 Authentication & Registration

Email-based registration with a verification code, login with BCrypt password verification, and logout that clears the user's location. Email verification codes are sent through Gmail SMTP and stored temporarily in the verification_codes table until consumed.

Authenticati
on &
Registration

Controller	com.DogMate.Controller.AuthController
Service	com.DogMate.Service.UserService com.DogMate.Service.PendingRegistrationService com.DogMate.Service.EmailVerificationService
Repository	IUserRepository → UserRepository IVerificationCodeRepository → VerificationCodeRepository PendingRegistrationRepository
Domain	UserAccount (base) → RegularUser, DogWalkerUser, AdminUser PendingRegistration, VerificationCode
Endpoints	POST /api/auth/login POST /api/auth/logout POST /api/auth/verify-registration POST /api/auth/resend-verification POST /api/users/register

5.2 Dog Profile Management

Creating, editing, listing, and deleting a user's dog profiles. Each dog is linked to one or more users through DogRelationship entities, which allows the same dog to be associated with multiple owners.

Dog Profile Management

Controller	<code>com.DogMate.Controller.DogController</code>
Service	<code>com.DogMate.Service.DogService</code>
Repository	<code>IDogRepository</code> → <code>DogRepository</code> <code>IDogRelationshipRepository</code> → <code>DogRelationshipRepository</code>
Domain	<code>Dog</code> , <code>DogRelationship</code> , <code>RelationshipType</code> , <code>DogProfile</code> , <code>HealthProfile</code> , <code>BehaviorProfile</code>
Endpoints	<code>GET /api/dogs</code> <code>GET /api/dogs/search</code> <code>POST /api/dogs/add</code> <code>PUT /api/dogs/{userId}/{dogId}</code> <code>GET /api/dogs/user/{userId}</code> <code>DELETE /api/dogs/{dogId}</code>

5.3 Vaccinations

Per-dog vaccination history with an automatic countdown to the next booster. Ownership of each record is verified through `DogRepository.findAllForRegularUser` before any create / update / delete operation.

Vaccinations

Controller	<code>com.DogMate.Controller.VaccinationController</code>
Service	<code>com.DogMate.Service.VaccinationService</code>
Repository	<code>DogVaccinationRepository IDogRepository → DogRepository</code>
Domain	<code>DogVaccination</code>
Endpoints	<code>GET /api/vaccinations/user/{userId} POST /api/vaccinations/user/{userId}</code> <code>PUT /api/vaccinations/{vaccinationId} DELETE</code> <code>/api/vaccinations/{vaccinationId}</code>

5.4 Medications

Per-dog medication and treatment history with countdown to the next dose. The medication autocomplete on the mobile client is powered by querying past entries for the same dog.

Medications

Controller	<code>com.DogMate.Controller.MedicationController</code>
Service	<code>com.DogMate.Service.MedicationService</code>
Repository	<code>DogMedicationRepository IDogRepository → DogRepository</code>
Domain	<code>DogMedication</code>
Endpoints	<code>GET /api/medications/user/{userId} POST /api/medications/user/{userId}</code> <code>PUT /api/medications/{medicationId} DELETE</code> <code>/api/medications/{medicationId}</code>

5.5 Food Inventory

Tracks current food stock per dog, daily intake requirements, and the calculated number of days remaining. The "I Bought a New Bag" button calls the `/renew` endpoint, which resets the remaining-days counter without re-entering the full record.

Food Inventory

Controller	<code>com.DogMate.Controller.FoodStockController</code> <code>com.DogMate.Controller.DogController</code> (legacy add-stock endpoint)
Service	<code>com.DogMate.Service.DogService</code> (food-stock methods)
Repository	<code>IFoodStockRepository</code> → <code>FoodStockRepository</code>
Domain	<code>FoodStock</code>
Endpoints	<code>GET /api/food-stock/user/{userId}</code> <code>PUT /api/food-stock/{id}/renew</code> <code>PUT /api/food-stock/{id}</code> <code>DELETE /api/food-stock/{id}</code> <code>POST /api/dogs/{dogId}/food-stock</code>

5.6 Reminders

Custom user reminders shown on the Home screen. Medication and vaccination reminders are auto-generated by the corresponding services and stored in the same Reminder table.

Reminders

Controller	<code>com.DogMate.Controller.ReminderController</code>
Service	<code>com.DogMate.Service.ReminderService</code> <code>com.DogMate.Service.UserService</code> (cache eviction)
Repository	<code>IReminderRepository</code> → <code>ReminderRepository</code>
Domain	<code>Reminder</code>
Endpoints	<code>POST /api/users/{userId}/reminders</code> <code>GET /api/users/{userId}/reminders</code> <code>DELETE /api/users/{userId}/reminders/{reminderId}</code>

5.7 Walks, Proximity & Ping

During an active walk, users share their live location via `UserService.updateUserLocation`. Nearby dog discovery is exposed through `UserController` endpoints, and "Propose Meeting" pings are delivered in real-time over WebSocket (STOMP). The `MatchingService` and `ProximityService` classes are skeletons reserved for future expansion of the matching algorithm.

Walks, Proximity & Ping

Controller	<code>com.DogMate.Controller.UserController</code> (location, pings) <code>com.DogMate.Controller.PingWebSocketController</code> <code>com.DogMate.Controller.ProximityController</code>
Service	<code>com.DogMate.Service.UserService</code> <code>com.DogMate.Service.SniffService</code> <code>com.DogMate.Service.MatchingService</code> <code>com.DogMate.Service.ProximityService</code>
Repository	<code>IUserRepository</code> → <code>UserRepository</code> <code>ISniffRequestRepository</code> → <code>SniffRequestRepository</code>
Domain	<code>RegularUser</code> (latitude/longitude fields), <code>Ping</code> , <code>SniffRequest</code> , <code>UserLiveLocation</code>

Endpoints	POST /api/users/ping POST /api/users/ping/respond GET /api/users/pings/pending/{userId} WS /ws-ping/** (STOMP)
------------------	--

5.8 Dog Walker Profile & Ratings

Professional walker profiles, working hours, service-area / city offerings, and the five-star rating system. The "Available Dog Walkers" listing for dog owners is served by the /available-with-professional-profile endpoint.

Dog Walker Profile & Ratings

Controller	com.DogMate.Controller.DogWalkerController
Service	com.DogMate.Service.DogWalkerService com.DogMate.Service.DogOwnerService
Repository	IDogWalkerRepository → DogWalkerRepository IDogWalkerRatingRepository → DogWalkerRatingRepository
Domain	DogWalkerUser, DogWalkerRating, WalkerCityOffering, OpeningHours
Endpoints	GET /api/dog-walkers/available-with-professional-profile GET /api/dog-walkers/{userId}/professional-profile PUT /api/dog-walkers/{userId}/professional-profile POST /api/dog-walkers/{walkerId}/ratings DELETE /api/dog-walkers/{walkerId}/ratings/{ratingId}

5.9 Emergency SOS — Clinic Finder

The SOS button locates the five nearest veterinary clinics using the user's GPS coordinates. Veterinary clinic data is stored in the `veterinary_clinics` table and seeded ahead of time. The `ClinicFinderService` class is currently a placeholder — the lookup is performed directly by the controller layer against the repository.

Emergency SOS — Clinic Finder

Controller	<code>com.DogMate.Controller.FinderController</code>
Service	<code>com.DogMate.Service.ClinicFinderService</code> (placeholder — to be implemented)
Repository	<code>IVeterinaryClinicRepository</code> → <code>VeterinaryClinicRepository</code>
Domain	<code>VeterinaryClinic</code> , <code>OpeningHours</code> , <code>EmergencyContact</code>
Endpoints	(currently inlined in <code>FinderController</code> — to be migrated to <code>ClinicFinderService</code>)

Tip: When extending the SOS feature, follow the same pattern as `VaccinationService` or `MedicationService`: keep ownership and validation in the service layer, expose only DTOs from the controller, and write the distance calculation in a static helper method on the service for unit testing.

Chapter 6

Cache Management

The application uses Caffeine — an in-process Java cache — to reduce repeated database roundtrips for frequently-read user data. Caching is enabled globally in DogMateApplication via the `@EnableCaching` annotation, and configured in `CacheConfig.java`.

Configured Caches

Cache Name	Stores	Evicted By
loggedUsers	Currently authenticated user lookups	Login, logout, password change, profile update, suspension, location update
dogsByUser	Per-user dog lists (for the Home screen)	Dog creation, deletion, and ownership changes
remindersByUser	Per-user reminder lists	Reminder creation, deletion, and bulk operations on the user

Tuning the Cache

The TTL is intentionally short (5 seconds) — it deduplicates rapid-fire requests from the mobile client without serving stale data. Adjust these values in `CacheConfig.java` only after measuring the effect on freshness for the affected endpoints.

```
// com.DogMate.Config.CacheConfig
@Bean
public CacheManager cacheManager() {
    CaffeineCacheManager cacheManager = new CaffeineCacheManager(
        "loggedUsers", "dogsByUser", "remindersByUser"
    );
    cacheManager.setCaffeine(
        Caffeine.newBuilder()
            .maximumSize(1000)
            .expireAfterWrite(5, TimeUnit.SECONDS) // tune carefully
    );
    return cacheManager;
}
```

Important: Cache eviction is annotation-driven. When adding a new method that mutates cached data, add the matching `@CacheEvict` (or `@Caching` with multiple `@CacheEvict` entries) to prevent stale data on subsequent reads. `UserService` methods provide complete examples of both forms.

Chapter 7

Security & CORS

Security configuration lives in `com.DogMate.Config.SecurityConfig`. The current setup is open by design: all `/api/**` endpoints, the H2 console, the WebSocket endpoint, and the Selenium test pages are accessible without authentication. Authentication of API calls is performed manually by the controller layer (via the `userId` path variable), not by Spring Security.

Current Open Paths

```
// SecurityConfig.securityFilterChain(...)
.authorizeHttpRequests(auth -> auth
.requestMatchers(
"/",
"/api/**",
"/h2-console/**",
"/ws-ping",
"/ws-ping/**",
"/selenium-test/**"
).permitAll()
.anyRequest().authenticated()
)
```

CORS — Allowed Origins

CORS is configured at the security-filter level. Add new front-end origins here when extending the app to additional domains. The list currently allows only local Vite ports (5173 and 4173).

```
config.setAllowedOrigins(Arrays.asList(
"http://localhost:5173",
"http://127.0.0.1:5173",
"http://localhost:4173",
"http://127.0.0.1:4173"
));
config.setAllowedMethods(Arrays.asList(
"GET", "POST", "PUT", "DELETE", "OPTIONS"
));
config.setAllowedHeaders(Arrays.asList("Authorization", "Content-Type"));
config.setAllowCredentials(true);
```

Password Hashing

Passwords are hashed with BCrypt using the PasswordEncoder bean defined in DogMateApplication. The same encoder is injected into UserService and used for both registration (encode) and login (matches). Do not introduce alternate hashing implementations — always use the injected PasswordEncoder.

```
// com.DogMate.DogMateApplication
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Chapter 8

Testing

The project ships with three layers of automated tests: JUnit 5 unit tests for domain and service logic, Spring Boot integration tests for repository and service wiring, and Selenium browser tests that drive the static HTML test pages under `resources/static/selenium-test/`.

Test Directory Layout

```
src/test/java/com/DogMate/  
+-- DogMateApplicationTests.java <- Spring context smoke test  
+-- util/PhoneValidationTest.java <- Pure utility unit tests  
+-- Domain/ <- Domain entity unit tests  
| +-- UserAccountTest.java  
| +-- RegularUserTest.java  
| +-- DogRelationshipTest.java  
+-- Service/ <- Service-layer integration tests  
| +-- UserServiceTest.java  
| +-- DogServiceTest.java  
| +-- DogWalkerServiceTest.java  
| +-- AdminInsertIntegrationTest.java  
| +-- FoodStockIntegrationTest.java  
+-- Controller/ <- MockMvc controller tests  
| +-- UserControllerTest.java  
| +-- DogWalkerControllerTest.java  
+-- selenium/ <- End-to-end browser tests  
+-- BaseSeleniumTest.java  
+-- LoginSeleniumTest.java  
+-- AdminSeleniumTest.java  
+-- AddDogSeleniumTest.java
```

Running Tests

1

Run all unit and integration tests

`./mvnw test` — executes everything under `src/test/java`.

2

Run a single test class

`./mvnw test -Dtest=UserServiceTest` — useful while iterating on a feature.

3

Run a single test method

`./mvnw test -Dtest=UserServiceTest#shouldRegisterNewUser`

4

Skip tests during build

`./mvnw clean install -DskipTests` — only when packaging for deployment.

5

Run Selenium tests

Requires Chrome and ChromeDriver. `BaseSeleniumTest` spins up the Spring Boot context and a headless browser automatically.

Tip: When adding tests for a service that interacts with a repository, use `@ActiveProfiles("memory")` (see Chapter 4) to run against the in-memory repository implementation. This makes test runs deterministic and removes the database dependency from CI.

Common Maintenance Tasks

Adding a New Domain Entity

1	Create the JPA entity Add a new class under <code>com.DogMate.Domain</code> with <code>@Entity</code> and <code>@Table</code> annotations.
2	Define the repository interface Add <code>IRepository</code> under <code>com.DogMate.Service</code> with the methods the service layer needs.
3	Implement the JPA repository Add <code>Repository</code> under <code>com.DogMate.Infrastructure</code> extending <code>JpaRepository</code> and the interface.
4	Create the service class Add <code>Service</code> under <code>com.DogMate.Service</code> . Inject the I-interface, never the JPA class.
5	Add the controller Add <code>Controller</code> under <code>com.DogMate.Controller</code> with <code>@RequestMapping("/api")</code> .
6	Wire DTOs Add request and response DTOs under <code>com.DogMate.DTO</code> . Never expose entities directly over the wire.
7	Add cache eviction if relevant Annotate mutating service methods with <code>@CacheEvict</code> against the matching cache name.
8	Write tests At minimum: a domain test, a service test, and a controller test using <code>MockMvc</code> .

Changing Database Credentials

Update the values in the `.env` file (or in the environment variables of the deployment platform). Do not commit credentials to `application.properties` — the existing hard-coded fallback values should be removed before any public release.

Rotating the Gmail App Password

1	Sign in to the dogmateteam@gmail.com Google account Use Account Security → 2-Step Verification.
2	Generate a new App Password Under "App passwords", create a new password labelled "DogMate Backend".
3	Update the environment variable Set <code>SPRING_MAIL_PASSWORD</code> in the deployment environment (or <code>.env</code> locally).
4	Restart the application The new password is picked up at startup. No code changes required.

Understanding the Startup Reset Behavior

On every application start, `StartupInitializer` runs `UserRepository.resetAllUsersOnStartup`, which sets `loggedIn` to false and clears latitude/longitude for every user. This guarantees that no user remains "stuck" as online after a server restart. If you need to disable this behavior temporarily (for example to investigate a stale-session bug), comment out the `@Component` annotation on `StartupInitializer`.

```
// com.DogMate.Config.StartupInitializer
@Override
@Transactional
public void run(String... args) {
    int updatedCount = userRepository.resetAllUsersOnStartup();
    logger.info("Reset {} users - all logged out and locations cleared",
        updatedCount);
}
```

This document is maintained alongside the codebase. When making structural changes to the project — adding new layers, switching frameworks, or introducing additional caches — update the relevant chapter so the next maintainer inherits an accurate map.